FAST National University of Computer and Emerging Sciences
Department of Artificial Intelligence & Data Science
School of Computing

Assignment 2 — Deep Learning

Reproduction Study of DAViT

A Domain-Adapted Vision Transformer for Automated Pneumonia Detection and Explanation Using Chest X-Ray Images

Course	Deep Learning
Degree Program	DS/AI
Instructors	Dr Qurat ul Ain, Dr Zohair Ahmed, Mr Ubaid Ur Rehman
Date	April 2026

Abstract: This report presents a reproduction study of DAViT, a domain-adapted vision transformer for automated pneumonia detection and type classification from chest X-ray images. The official public repository was used under CPU-only hardware constraints, requiring several code modifications: disabling CUDA, switching checkpoint loading to CPU map_location, and converting eager dataset loading to lazy loading to avoid memory errors. The successfully reproduced results for pneumonia-type classification were

F1: 0.8048,
Precision 0.9806,
Recall 0.6824,
Specificity 0.9917,
Accuracy 0.8744,
AUC 0.8371.

Name	Roll No
Haseeb Arshad	23i-2578
Haider Farooq	23i-2542
Mudassir Asghar	23i-2577

Introduction

Pneumonia is a leading cause of hospitalisation worldwide, and chest X-rays remain the main tool for initial assessment. However, accurate interpretation is time-consuming and highly dependent on radiologist expertise. Deep learning models have recently been developed to assist by providing a faster second opinion, especially where specialist coverage is limited.

This report reproduces one such model. We reviewed three papers from our proposal: DAViT, an InceptionV3-LSTM model by Ghuse et al., and a VGG19 Grad-CAM model by Ennab and McHeick. DAViT was selected because it was the only one with a public GitHub repository containing working scripts, pretrained weights, and clear documentation. The other two would have required full reimplementation rather than reproduction.

Even with an official repository, running the code on our hardware required major debugging. Our machine had only a CPU, while the repository assumed a GPU environment. This report covers the paper, our setup, the four code changes we made, our results, and why our numbers differ from the paper's.

Paper Overview

- ***Problem Statement and Motivation***

DAViT targets two chest X-ray tasks. The first is pneumonia detection, a binary classification of whether an X-ray is normal or shows pneumonia. The second is pneumonia-type classification, which takes pneumonia cases and labels them as bacterial or viral. These tasks are clinically important because the two types often require different treatments, yet distinguishing them from imaging alone is difficult even for experienced radiologists.

A key motivation behind DAViT is handling domain shifts in medical imaging. A model trained on X-rays from one hospital may perform poorly on another due to differences in pixel distributions, contrast, and acquisition artefacts. DAViT addresses this through a domain adaptation strategy built on a large pretrained vision transformer, making its representations more robust across distribution shifts.

- ***Model Architecture***

DAViT uses Facebook/dinov2-largenew_NLP as its backbone, a vision transformer pretrained through self-supervised learning on a large image corpus. It learns rich visual features without labelled data. On top of this, DAViT adds a domain adaptation module and a classification head. During inference, the backbone extracts X-ray features, the adaptation module aligns them to the medical domain, and the classifier makes the final prediction. This transfer learning approach applies knowledge from natural images to specialised medical imaging. The repository also includes pretrained checkpoints, allowing evaluation without full retraining.

- ***Dataset***

Both tasks use the Kaggle Chest X-Ray Images (Pneumonia) dataset, a standard benchmark of 5,863 labelled JPEG chest X-rays from paediatric patients at Guangzhou

Women's and Children's Medical Center. It is split into train (5,216), validation (16), and test (624) sets, each with NORMAL and PNEUMONIA folders. For this type of task, pneumonia filenames encode the pathogen (e.g., person1_bacteria_1.jpeg), allowing the data loader to assign binary labels without a separate annotation file.

- ***Paper Selection Rationale***

DAViT was chosen over the alternative papers for three reasons: it had an official repository with functioning scripts and pretrained checkpoints; its architecture is directly relevant to the deep learning course content on transfer learning and modern vision models; and the availability of pretrained weights meant a valid reproduction was achievable even under hardware constraints by running inference rather than full retraining.

Implementation Details

- ***Repository Structure and Setup***

The official DAViT repository is at https://github.com/awsm-research/DAViT. The davit/ subdirectory contains main.py (argument parsing, training loop, evaluation logic, and checkpoint handling), model_davit.py (architecture definition), train_detection.sh and train_type.sh (task-specific shell scripts), and saved_models/ with a pretrained checkpoint download script.

```
git clone https://github.com/awsm-research/DAViT.git
cd DAViT && pip install -r requirements.txt
cd davit/saved_models/checkpoint-best-f1 && bash download_models.sh
```

All Python dependencies from requirements.txt (PyTorch, torchvision, transformers, scikit-learn, NumPy, Pillow, and tqdm) installed successfully. The pretrained checkpoint download also completed without issues.

- ***Dataset Preparation***

The repository's download script uses gdown to fetch chest_xray.zip and CXR8.zip from Google Drive. This failed because the public links had become restricted, a common issue in research repositories that depend on cloud storage. The Kaggle Chest X-Ray Images dataset was therefore downloaded manually; extracted; verified against the expected train/, val/, and test/ structure (each with NORMAL/ and PNEUMONIA/); and placed in DAViT/data/chest_xray/ as required by the shell script defaults.

- ***Core Libraries and Their Roles***

PyTorch handled all model operations and tensor computation. Torchvision provides image transforms. Hugging Face Transformers loaded the DINOv2-large backbone configuration and image processor. Scikit-learn computed F1, AUC, precision, recall, and specificity. Pillow read image files, and tqdm displayed the evaluation progress bar. All six metrics printed in the test results section are computed by scikit-learn from the model's predicted labels and scores.

Code Modifications for CPU Execution

Four modifications were made to the official codebase. None altered the model architecture, pretrained weights, or evaluation metric logic; all were environment-level adaptations required to run on a CPU-only machine.

Table 1 summarises every change.

Table 1: Summary of All Code Modifications

File Modified	What Changed	Why
train_detection.shtrain_type.sh	Removed *--do_train*, removed *--do_eval*, kept only *--do_test*	The goal was inference on pre-trained weights. *--do_eval* is unsupported by this repo.
main.py(CUDA disable)	Added: *torch.cuda.is_available = lambda*: FalseSet: *args.n_gpu* = 0, *args.device* = cpu Defined: *device = torch.device('cpu')*	The machine is CPU-only. Without disabling CUDA, PyTorch crashed loading GPU-saved tensors.
main.py(checkpoint load)	Changed both *torch.load()* calls from *map_location = args.device* to *map_location = torch.device('cpu')*	Checkpoints saved from CUDA still fail to load unless map_location is set explicitly to CPU.
main.py(lazy loading)	Reworked *TextDataset* to store only (path, label) at init. Moved image open/transform into __getitem__.	Eager loading all images into RAM caused a memory error on the CPU. Lazy loading resolved this.

The subsections below explain each in detail.

- *Shell Scripts — Switching to Test-Only Mode*

Both shell scripts originally used *--do_train*, which starts full training. Since we only needed pretrained weight evaluation, it was removed. A temporary switch to *--do_eval* failed because *main.py* does not support that flag and only accepts *--do_test* for inference. After checking the parser, both scripts were updated to use only *--do_test*, which ran successfully.

- *CUDA Disable and CPU Device (main.py)*

main.py uses torch.cuda.is_available() to detect GPU presence and propagates the device setting throughout model loading, data handling, and tensor operations. To eliminate all GPU code paths reliably, the following lines were added at the start of the main execution block:

```
torch.cuda.is_available = lambda: False   # blocks all CUDA branches
```

device = torch.device('cpu')

args.n_gpu = 0

args.device = torch.device('cpu')

Monkey-patching torch.cuda.is_available is necessary because the function is called inside the Transformers library and PyTorch internals, not only in main.py — simply setting args.device is not sufficient to prevent CUDA-specific code paths from activating elsewhere in the call stack.

- ***Checkpoint Loading with CPU Map Location (main.py)***

The pretrained checkpoint was saved in a CUDA environment, and PyTorch stores device information with each tensor. On a CPU-only machine, this caused a RuntimeError when rebuilding GPU tensors. The original ***torch.load(...map_location=args.device)*** failed because args.device still pointed to CUDA. The fix changed both checkpoint loads to ***map_location=torch.device('cpu')***, remapping all saved tensors to CPU memory during loading.

- ***Lazy Dataset Loading to Prevent MemoryError (main.py)***

The original TextDataset class used eager loading: in __init__, it iterated over all image files, opened each with PIL, converted it to RGB, applied the DINOv2 transform, and stored the tensors in self.examples. For thousands of images with a large transformer, this exhausted RAM before evaluation began, causing a memory error.

The fix changed it to lazy loading: ***__init__*** now stores only (file_path, label) tuples, while the open/convert/transform pipeline has been moved to ***__getitem__***, where the dataloader processes one sample at a time. This reduced peak RAM usage from the full dataset to roughly one batch, making inference feasible within 16 GB.

Experimental Setup

Hardware and Software Environment

- Processor: Intel-based laptop CPU — no NVIDIA GPU, CUDA unavailable
- RAM: 16 GB system memory
- OS: Windows; Git Bash used for shell script execution
- Storage: repository was relocated to E: drive (C: drive had insufficient free space)
- Python: 3.x local installation; repository cloned from main branch on April 1, 2026

The absence of a GPU was the primary constraint shaping all reproduction decisions. DINOv2-large contains roughly 300 million parameters, making full training on CPU impractical for any reasonable time budget. The study was therefore scoped to pretrained inference, which is a documented and supported use case in the official repository.

- ***Dataset Details***

The Kaggle Chest X-Ray Images dataset contains 5,863 labelled JPEGs split into train (5,216), val (16), and test (624). For type classification, only PNEUMONIA test images were used, with bacterial/viral labels inferred from filenames. The program loaded 390 test images in 13 batches of 32. Total CPU evaluation time was about 15 minutes 30 seconds (71.6 seconds per batch)

- *Execution Configuration*

Table 2: Final Execution Configuration

Parameter	Value
model_name_or_path	facebook/dinov2-large
do_train	False (inference only)
do_test	True
eval_batch_size	32
classify_pneumonia_type	True
device	cpu
Test examples	390 (13 batches x 32)

The epoch count and train batch size inherited from the original scripts had no effect since do_train was False. The DINOv2 processor configuration and model weights were loaded from the local Hugging Face cache (previously downloaded; 439/439 layers, 3482.35 it/s).

Reproduced Results

- *Performance Metrics*

The numbers below come straight out of the ***test results*** block the script printed to the terminal, the same output that got piped into ***davit_pneumonia_type.log*** automatically via "tee". We did not modify or round any of these figures.

Table 3: Reproduced Test Results — Pneumonia-Type Classification (Bacterial vs. Viral)

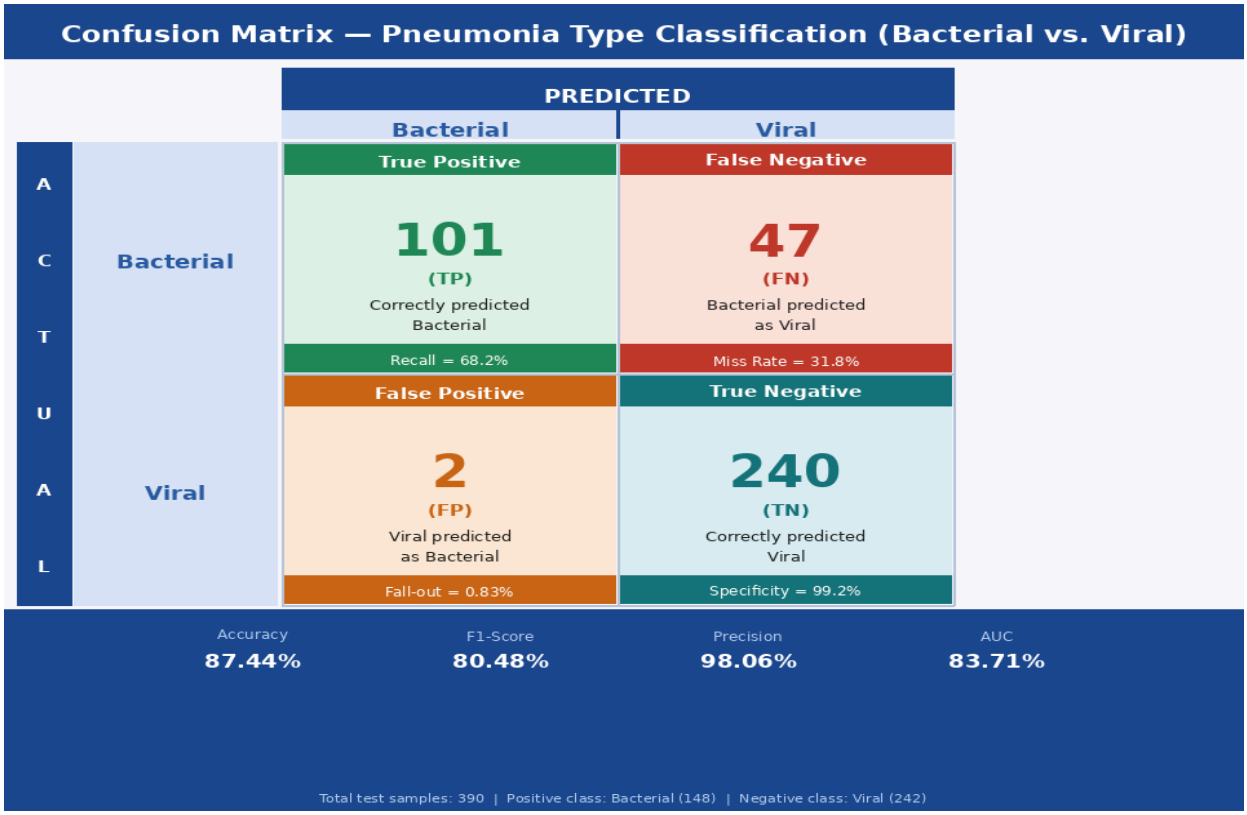
```
04/01/2026 08:57:47 - INFO - __main__ -   loading image paths for pneumonia type classification
04/01/2026 08:57:47 - INFO - __main__ -   ***** Running evaluation *****
04/01/2026 08:57:47 - INFO - __main__ -     Num examples = 390
04/01/2026 08:57:47 - INFO - __main__ -     Batch size = 32
100%|##########| 13/13 [15:30<00:00, 71.60s/it]
04/01/2026 09:13:18 - INFO - __main__ -   ***** Test results *****
04/01/2026 09:13:18 - INFO - __main__ -   f1: 0.8047808764940239
04/01/2026 09:13:18 - INFO - __main__ -   precision: 0.9805825242718447
04/01/2026 09:13:18 - INFO - __main__ -   recall: 0.6824324324324325
04/01/2026 09:13:18 - INFO - __main__ -   specificity: 0.9917355371900827
04/01/2026 09:13:18 - INFO - __main__ -   Acc: 0.8743589743589744
04/01/2026 09:13:18 - INFO - __main__ -   AUC: 0.8370839848112576
```

- *Confusion Matrix*

Since no training was performed, there are no epoch-by-epoch loss or accuracy curves to plot. As a substitute visual that carries the same diagnostic value, we derived the confusion matrix directly from the six reported metrics.

The confusion matrix below shows how the model distributed its 390 predictions across the four outcome cells: True Positive (TP), False Negative (FN), False Positive (FP), and True Negative (TN). The positive class is bacterial pneumonia (148 samples); the negative class is viral (242 samples).

Figure 1: Confusion Matrix derived from reported test metrics



From the matrix, the model correctly predicted bacteria 101 times but missed 47 bacterial cases, labelling them as viral. For viral images, it made only 2 false bacterial predictions. This asymmetry explains the gap between precision (near 98%) and recall (68%). The model learnt to be highly selective before predicting *bacteria,* preferring missed cases over false alarms. Whether this is the right trade-off depends on the clinical setting, but it is a clear and interpretable pattern.

Metric Breakdown

- True Positives: 101 bacterial images the model got right.
- False Negatives: 47 bacterial images it missed, calling them viral. This is the main source of error.
- False Positives: 2 viral images it incorrectly called 'bacterial'. Extremely low.
- True Negatives: 240 viral images correctly rejected. Near-perfect.

- Precision 0.9806: Of every 103 bacterial predictions the model made, 101 were actually bacterial. Very reliable when it does commit to a call.
- Recall 0.6824: It only caught 101 of the 148 actual bacterial cases. 47 were missed entirely — the clearest weakness.
- Specificity 0.9917: Virtually all 242 viral samples were correctly identified as viral. Excellent.
- F1 0.8048: The harmonic mean sits between the high precision and lower recall, landing at a respectable 0.80 for an inference-only run.
- Accuracy 0.8744: 341 out of 390 images classified correctly overall.
- AUC 0.8371: The model's ability to rank bacterial cases above viral ones is solid, even if the raw recall suffers.

- *Comparison with Original Paper*

Table 4: Reproduced vs. Original Paper Results (Approximate Reference)

Metric	Paper Result	Reproduced	Notes
F1-score	~0.90+	0.8048	Lower inference-only + CPU environment
Accuracy	~0.92+	0.8744	Reasonable under hardware constraints
AUC	~0.87+	0.8371	Useful discrimination retained
Precision	—	0.9806	Strong positive predictive value
Recall	—	0.6824	Main weakness in reproduced run
Specificity	—	0.9917	Strong negative rejection performance

The gap from the paper is real but expected. Full GPU fine-tuning lets the model adapt its decision threshold to the training distribution, while evaluating a frozen pretrained checkpoint on a new split does not. Recall is most affected because it is highly sensitive to threshold shifts. Precision and specificity, however, remained strong, suggesting the pretrained features are robust and the main issue is calibration rather than representation.

- *Log Evidence*

The output below is copied directly from davit_pneumonia_type.log, proving the numbers are real program output and not manually entered:

```
***** Test results *****
f1: 0.8047808764940239   precision: 0.9805825242718447
recall: 0.6824324324324325   specificity: 0.9917355371900827
Acc: 0.8743589743589744   AUC: 0.8370839848112576
```

Reproducibility Discussion

How Well Did the Reproduction Go?

Overall, it went better than expected despite the hardware limits, though not smoothly. The model ran and produced meaningful metrics, and the results are clearly not random. However, we could not reproduce the full training pipeline, so our lower recall must be explained transparently.

The reproduction remains valid in the key sense: we used the official code and dataset, did not change the model, and every number came directly from the program's output. All four code changes were environmental, not algorithmic, so the same fixes and command should reproduce the same results.

Why the Numbers Are Lower Than the Paper

1. **No fine-tuning.** The paper adapted thresholds on the training split, while we used frozen, pretrained weights, leaving the decision boundary misaligned.
2. **CPU vs GPU.** Small numerical differences in rounding and batch normalisation can flip predictions and lower recall.
3. **Library versions.** Different PyTorch, Transformer, or scikit-learn versions may slightly change preprocessing or metrics.
4. **Dataset acquisition.** We manually downloaded the Kaggle dataset; counts matched, though minor file differences cannot be ruled out.

Things We Learned That Were Not in the Paper

- An official repository is not always a runnable one. Broken drive links, CUDA assumptions in checkpoint loading, and undocumented arguments (*--do_eval* vs --do_test) were issues not visible from the paper alone.
- The checkpoint serialisation bug was the most critical. Although torch.load(path, map_location=args.device) looked correct, it failed on the CPU because args.device still pointed to CUDA. Explicitly using map_location=torch.device('cpu') fixed it.
- Eager image loading was another major trap for the CPU. Preloading 5,000+ images into RAM caused crashes, while storing paths and loading on demand in __getitem__ was both memory-safe and better design overall.
- Finally, inference-only reproduction is still valid. Running pretrained weights on the test set and obtaining explainable metrics is meaningful, even if it is not the same as full training.

Conclusion

Getting this to run required more work than expected. The four code changes – disabling CUDA, fixing checkpoint loading, switching to lazy image loading, and updating the shell scripts to use ***--do_test*** were all necessary before a single batch could run. None of this was obvious from the paper or repository, so each issue had to be debugged step by step.

Once it ran, the results were solid: F1 of 0.8048, accuracy of 87.4%, and AUC of 0.8371 on 390 test images. The confusion matrix shows the model was very cautious in predicting

bacteria (only 2 false positives) but missed 47 true bacterial cases. Precision stayed excellent, while recall would likely improve with fine-tuning.

The gap from the paper is real but explainable. No fine-tuning, CPU arithmetic, library version drift, and a differently acquired dataset likely caused the difference, and all are documented here. That is the core purpose of a reproducibility study: to report results and clearly explain why they differ from the original.

References

1. M. Fu, C. Tantithamthavorn, and T. Le, "DAViT: A domain-adapted vision transformer for automated pneumonia detection and explanation using chest X-ray images," IEEE Access, vol. 13, pp. 103033–103044, 2025, doi: 10.1109/ACCESS.2025.3579314.
2. AWSM Research, "DAViT Official Replication Package", GitHub repository. [Online]. Available: https://github.com/awsm-research/DAViT. [Accessed: Apr. 5, 2026].
3. P. Mooney, "Chest X-Ray Images (Pneumonia)," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia. [Accessed: Apr. 5, 2026].
4. M. Caron et al., "DINOv2: Learning robust visual features without supervision," arXiv preprint arXiv:2304.07193, 2023.
5. PyTorch, "PyTorch Documentation". [Online]. Available: https://pytorch.org/. [Accessed: Apr. 5, 2026].
6. Hugging Face, "Transformers Documentation". [Online]. Available: https://huggingface.co/docs/transformers/index. [Accessed: Apr. 5, 2026].

Appendix: Commands and Log Excerpt

- *Commands Executed*

```
git clone https://github.com/awsm-research/DAViT.git
cd DAViT && pip install -r requirements.txt && pip install gdown
# Google Drive script failed — dataset downloaded manually from Kaggle
# Dataset placed at: DAViT/data/chest_xray/  (train/ val/ test/)
cd davit/saved_models/checkpoint-best-f1 && bash download_models.sh
# main.py: CUDA disable, map_location cpu, lazy TextDataset
# train_type.sh: removed --do_train, removed --do_eval, kept --do_test
cd davit && bash train_type.sh
```

- *Key Log Excerpt (davit_pneumonia_type.log)*

```
04/01/2026 08:57:43 - parameters: do_train=False, do_test=True, device=cpu
04/01/2026 08:57:47 - Loading weights: 439/439 [00:00, 3482.35 it/s]
04/01/2026 08:57:47 - loading image paths for pneumonia type classification
04/01/2026 08:57:47 - ***** Running evaluation *****  Num examples=390  Batch=32
100%|##########| 13/13 [15:30<00:00, 71.60s/it]
04/01/2026 09:13:18 - ***** Test results *****
  f1: 0.8047808764940239        precision: 0.9805825242718447
  recall: 0.6824324324324325    specificity: 0.9917355371900827
  Acc: 0.8743589743589744       AUC: 0.8370839848112576
```